

Git é um software de versionamento, ou seja, a cada alteração que você faz no código, o git criará uma cópia do estado que o arquivo se encontra no momento com informações como data, quem modificou e um breve resumo do que foi alterado, possibilitando assim voltar no momento que você salvou.

O git foi desenvolvido em 2005, pelo mesmo criador do Kernel do linux, que no caso foi Linus Torvalds com o intuito de ajudar no versionamento do código do próprio linux. Ademais, seu intuito também era ter contribuições de códigos ao redor do mundo. O git é um software open source, ou seja, código aberto e multiplataforma.

Antigamente era utilizado master como branch default no nosso repositório. Hoje em dia, o pessoal utiliza como padrão: main.

Foi trocado porque master se remete a master and slave, ou seja, mestre e escravo. O pessoal passou a não gostar e definiram main como branch default, que significa principal.

commit - salva todas as alterações realizadas no código até então.

Para ficar mais fácil, basta imaginar um jogo. Quando chegamos em uma parte específica e salvamos o progresso, estamos realizando um commit.

Outro exemplo seria um trabalho de pesquisa em grupo, onde há introdução, desenvolvimento e conclusão. Considere que finalizei a introdução, agora, quando eu der um commit, vou salvar tudo que fiz até o momento.

Se no outro dia eu for fazer outra introdução, apagando a anterior e eu não gostar do resultado final, poderei voltar ao trabalho no momento em que realizei o commit da modificação antiga.

Uso do bash

Quando abrimos o bash diretamente pelo software, ele inicia na pasta home, que no caso, é o início.

#####comandos bash

ls - listagem (mesma coisa do dir no cmd. Por sinal, dir tbm funciona)

dir - lista tudo como o ls, porém o aspecto é mais primitivo.

cd - navegação (mesma lógica do cmd)

pwd - mostra o caminho absoluto que me encontro, ou seja, o diretório completo

ls -a - vai listar todos os arquivos, incluindo os ocultos. Detalhe, vai aparecer '.', onde esses dois pontos é o main da pasta respectiva. Ademais '.' apenas um ponto, é apenas uma referência da própria

cd .. - volto uma pasta antes(é tipo o cd.. do cmd, porém separados.)

clear - limpa o terminal

ctrl + l - também limpa o terminal

mkdir - criar uma nova pasta (mkdir nome_da_pasta)

rm - remover arquivo (rm - nome_do_arquivo.txt)

rm -r nome_da_pasta(remover pasta e subpastas)

rm -rf vai excluir uma pasta e ao mesmo tempo, força a sua exclusão

rm --help - verifica quais possibilidades de execução eu tenho para o comando apagar.
(funciona para os demais comandos também)

Git é um software de versionamento, ou seja, a cada alteração que você faz no código, o git criará uma cópia do estado que o arquivo se encontra no momento com informações como data, quem modificou e um breve resumo do que foi alterado, possibilitando assim voltar no momento que você salvou.

O git foi desenvolvido em 2005, pelo mesmo criador do Kernel do linux, que no caso foi Linus Torvalds com o intuito de ajudar no versionamento do código do próprio linux. Ademais, seu intuito também era ter contribuições de códigos ao redor do mundo. O git é um software open source, ou seja, código aberto e multiplataforma.

Antigamente era utilizado master como branch default no nosso repositório. Hoje em dia, o pessoal utiliza como padrão: main.

Foi trocado porque master se remete a master and slave, ou seja, mestre e escravo. O pessoal passou a não gostar e definiram main como branch default, que significa principal.

commit - salva todas as alterações realizadas no código até então.

Para ficar mais fácil, basta imaginar um jogo. Quando chegamos em uma parte específica e salvamos o progresso, estamos realizando um commit.

Outro exemplo seria um trabalho de pesquisa em grupo, onde há introdução, desenvolvimento e conclusão. Considere que finalizei a introdução, agora, quando eu der um commit, vou salvar tudo que fiz até o momento.

Se no outro dia eu for fazer outra introdução, apagando a anterior e eu não gostar do resultado final, poderei voltar ao trabalho no momento em que realizei o commit da modificação antiga.

Uso do bash

Quando abrimos o bash diretamente pelo software, ele inicia na pasta home, que no caso, é o início.

#####comandos bash

ls - listagem (mesma coisa do dir no cmd. Por sinal, dir tbm funciona)

dir - lista tudo como o ls, porém o aspecto é mais primitivo.

cd - navegação (mesma lógica do cmd)

pwd - mostra o caminho absoluto que me encontro, ou seja, o diretório completo

ls -a - vai listar todos os arquivos, incluindo os ocultos. Detalhe, vai aparecer '.', onde esses dois pontos é o main da pasta respectiva. Ademais '.' apenas um ponto, é apenas uma referência da própria

cd .. - volto uma pasta antes(é tipo o cd.. do cmd, porém separados.)

clear - limpa o terminal

ctrl + l - também limpa o terminal

mkdir - criar uma nova pasta (mkdir nome_da_pasta)

rm - remover arquivo (rm - nome_do_arquivo.txt)

rm -r nome_da_pasta(remover pasta e subpastas)

rm -rf vai excluir uma pasta e ao mesmo tempo, força a sua exclusão

rm --help - verifica quais possibilidades de execução eu tenho para o comando apagar.
(funciona para os demais comandos também)

mv - move arquivos de uma pasta para outra, no entanto, se eu digitar o arquivo existente e um arquivo não existente, ele renomeia
mv arquivo_1 arquivo_2 #renomeou arquivo_1 para arquivo_2
printf - printa algo na tela (printf"ola mundo")
touch - cria um arquivo (touch arquivo.txt)
start - inicia alguma coisa (start arquivo.txt)
cat - mostra o conteúdo de um arquivo de texto (cat arquivo.txt)

####

lembrando que o . faz referência a própria pasta. Ou seja, se houver algum comando com apenas um PONTO, significa que o cara está falando da pasta que ele se encontra no momento

####

#####inicialização do git:

O primeiro passo é ir para a pasta que será utilizada para o projeto. Após isso, no bash, digite:

git init .

Após digitar o comando acima, uma pasta denominada git será incrementada. Esta pasta será oculta, então se você quiser ver ela, precisará digitar ls -a

#agora, essa pasta é especial, pois se tornou um repositório git que é capaz de versionar coisas

#####comandos git

git status - lista em verde todas as pastas e arquivos que vão ser salvos no commit. Se estiver vermelho, significa que esses arquivos ainda não estão no commit.

git add nome_do_arquivo - insere algum arquivo em vermelho que será adicionado na lista para ir no commit

git config --global user.email "seuemail@gmail.com" - define um email padrão para você, ou seja, quando você commitar, será esse email que irá aparecer.

git config --global user.name "Seu nome" - define um nome para você e quando você commitar, vai aparecer seu nome também.

git commit - m "descreva o que foi mudado no código" - faz um commit, ou seja, salva o progresso atual. o -m vai inserir uma mensagem descritiva, informando a mudança.

##Após realizar o commit, ele será printado na tela com o hash (identificador único, como se fosse o cpf do commit) e informará também a descrição que você forneceu. Ademais, será informado o número de arquivos que foram alterados, o que foi removido e o que foi adicionado.

git config --global user.email - caso o email já tenha sido digitado, ele informará o email que você forneceu anteriormente (o mesmo serve para o nome de usuário)

git status - também utilizado para verificar se há algo novo para ser implementado no commit. Se houver, ficará verde ou vermelho, se não houver, informará que não tem nada para ser comitado em inglês.

#Agora, o commit foi salvo e ele se encontrará na pasta oculta git. O commit também pode ser chamado de snapshot, que é uma cópia instantânea e completa de um sistema, arquivo ou dados em um momento específico.

nano comandos git.txt - abre um editor no próprio bash. Se o arquivo existir, ele entra no arquivo, se não, ele cria um e abre o editor.

##

ctrl + o para salvar o arquivo no nano

ctrl + x para sair do nano

##

em um commit, posso fazer o save de mais de uma arquivo. Por exemplo, posso fazer o add em um arquivo e logo após fazer em outro. Após isso, faço o commit.

git log - lista todos os commits realizados de ordem crescente a partir da data e hora que foi commitado

se eu der um git log terá:

\$ git log

commit 17f7f70056cf673ecc7cd15d10eb57de249c6105 (HEAD -> main) (essa numeração gigantesca é o id do commit)

Author: GhostEasyX <jhonoliveirapsh@gmail.com> (tem o nome do editor e o email)

Date: Mon Mar 31 19:35:14 2025 -0300 (tem data e hora tbm)

tem mais anotações no comando bash e agora tem anotações de comandos git
#ademais, aparece a descrição

git add . - como dito antes, o ponto faz referência a pasta, ou seja, ele vai adicionar a pasta no commit.

git add * - como vimos em python, o * significa all, ou seja, tudo. Logo, se eu fizer um add *, ele vai adicionar todos os elementos da pasta no commit

Na prática, o ponto e o asterisco não tem diferença, pois fazem as mesmas coisas, porém tem conceitos diferentes. Enquanto um faz referência a própria pasta, o outro faz referência aos elementos da pasta. Mas, se você quiser dar uma utilidade maior para o *, você pode:

git add comando_* - todos os arquivos que tiver "comando_" em seu nome, por causa do *, será adicionado na lista do commit, na fila do commit.

#####Fluxo de trabalho:

1. criar o diretório do projeto
2. iniciar o git dentro do diretório criado (git init)
3. editar ou criar arquivos
4. git status
5. git add nome_do_arquivo
6. git status
7. git commit -m "mensagem descritiva"
8. git status
9. volta para o passo 3 e segue sucessivamente

#####Finalização aula 2

#####